Ders 3 Genişletilmiş Notu: Factory Method, Builder, Prototype ve Singleton Kalıpları

Ders 3 Genişletilmiş Notu: Factory Method, Builder, Prototype ve Singleton Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Kısa Tekrar ve Factory Method'a Giriş
- 2. Factory Method: Amaç, Örnek ve Sorun
- 3. Örnek Çözüm: Document ve Application
- 4. Factory Method'un Yapısı ve Uygulanabileceği Durumlar
- 5. Uygulama Örneği: E-Ticaret Sitesi
- 6. Kod Ayrıntıları: Annotation'lar ve Override
- 7. Soru Üzerine: Kalıtım ve Çok Biçimlilik Nasıl Çalışır?
- 8. Builder: Amaç, Örnek ve Çözüm
- 9. Builder'ın Yapısı ve Factory Kalıplarından Farkı
- 10. Uygulama Örneği: Araç Bayisi
- 11. Prototype: Amaç, Örnek ve Yapı
- 12. Sığ Kopya ve Derin Kopya
- 13. Uygulama Örneği: Oyun İçin Level Editörü
- 14. Singleton
- Örnek Sorular

1. Kısa Tekrar ve Factory Method'a Giriş

Geçen derste yoğun bir örnek olarak Abstract Factory kalıbı konuşulmuştu: hem kalıp anlatılmış hem de bir örnek üç farklı biçimde çözülüp avantajları ve dezavantajları tartışılmıştı. Hoca biraz daha açılarak kalıtımın yanlış kullanıldığı birkaç başka tasarımı da göstermişti. Bu derste yaratımsal (creational) kalıpların kalanına geçildi: Factory Method, Builder, Prototype ve Singleton.

Hoca önce bir terim notu verdi. "Factory" kelimesi geçtiğinde söz konusu olan, nesne üretimiyle ilgili bir kalıptır. Bu terim yerleşmiştir; tasarım kalıpları dışında herhangi bir yerde "factory" kelimesini görürseniz büyük ihtimalle nesne oluşturmak için kullanılan bir yöntemden söz ediliyordur.

2. Factory Method: Amaç, Örnek ve Sorun

Slayttaki tanım (ekrandan, 00:10:00) şöyledir:

- Amaç: Bir nesne oluşturmak için öyle bir ara yüz sunmak ki, oluşturulacak nesnenin sınıfına bu ara yüzü sağlayan sınıfın alt sınıfları karar verebilsin.
- Örnek: Farklı belge türleri ile çalışabilecek uygulamalar geliştirmeyi sağlayan bir çerçeve program (framework) hazırlanıyor. Temel sınıflar Belge ve Uygulama. Kullanıcı bu iki sınıftan kalıtım ile yeni sınıflar türeterek kendi yazılımını hazırlıyor.

- Sorun: Bir belge oluşturmak gerekince (aç, yeni vb. komutlar) çerçeve program bu işlemi nasıl yapacak? Uygulama, kullanıcının türeteceği yeni belge sınıfları hakkında önceden bilgi sahibi olamaz.

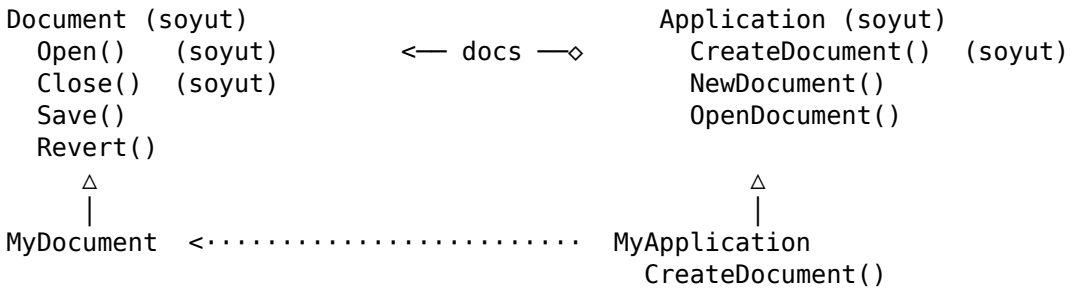
Kısacası bir istemci bir nesne oluşturma talebinde bulunacak, ama oluşan nesnenin özgül alt türüne arayüzü sunan taraf karar verecek.

Hoca framework kavramını hatırlattı. Framework bir anlamda "yarım yazılım", bir çerçeve yazılımdır. Size bazı temel yetenekler sağlar; siz bu yeteneklerin üstüne kendi iş mantığınızı ekleyerek uygulama hazırlarsınız. Programcıların hazırlayabileceği uygulama çeşitleri sınırsız olduğu için framework'lerde önemli bir esneklik ihtiyacı vardır: öngörülmeyen ihtiyaçlara nasıl ve ne kadar verimli yanıt verilebileceği önemlidir. Bu yüzden framework örnekleri hem GoF kitabında kalıbı tanıtmak için hem de örnek uygulamalarda sık kullanılır.

Örnekteki ihtiyaçlara bakınca iki temel kavram vardır: belge ve uygulama. Framework'ün kullanıcıları bu iki sınıftan kalıtımla yeni alt sınıflar türeterek kendi uygulamalarını yazarlar. Temel problem şudur: her belgenin oluşturulması, açılması, kaydedilmesi, yeni boş belge oluşturulması gibi komutlar vardır. Bunlar belgenin türünden bağımsızdır; metin dosyası, grafik dosyası ya da XML dosyası olsun, bu işler yapılmalıdır. Ama framework'te kodlanan uygulama kısmı, kullanıcının türeteceği yeni belge sınıflarını önceden tahmin edemez. Hocanın deyişiyle "kâhin değiliz". Buna rağmen ortak işler nasıl kodlanacak ve bu ortak adımlar farklı türleri nasıl destekleyecek? Problem budur.

3. Örnek Çözüm: Document ve Application

Dörtlü Çete'nin (GoF) verdiği örnek çözüm slayttadır (ekrandan, 00:12:00). Hoca ekranda bir çizgi çekerek şemayı ikiye ayırdı: çizginin üstü framework, altı framework kullanılarak geliştirilen uygulamadır.




```
Application.NewDocument():
    Document* doc = CreateDocument();
    docs.Add(doc);
    doc->Open();

MyApplication.CreateDocument():
    return new MyDocument
```

Temel uygulama sınıfı `Application` soyuttur. Hoca geçen haftaki notu hatırlattı: GoF şemalarında italik yazılan sınıflar ve metotlar soyuttur, ama hoca kendi örneklerinde «`abstract`» stereotipini yazmayı tercih ediyor. (Stereotip işaretindeki "iki küçüktür" ve "iki büyüktür" aslında tek karakterdir: « ve ».)

`Application` ile `Document` arasında parça-bütün ilişkisi vardır: uygulama bütün, belgeler parçadır. Soyut `Document` sınıfında açma ve kapama için soyut metotlar (`Open`, `Close`) ve kaydetme ile geri alma (undo) için somut metotlar (`Save`, `Revert`) vardır. Ayrım şöyle yapılmıştır: bir belge açılacağı zaman ekrana çizilmesi gerekir ve bu belgenin türüne çok bağlıdır. Kapatılırken de türe göre yapılması gereken farklı işler olabilir. Ama kaydetmede özel bir şey yoktur; sonuçta belge byte byte diske yazılır. Geri alma da framework'ün sağladığı ortak bir yetenektir; bunun için örneğin ileride göreceğimiz Memento kalıbı kullanılabilir.

Alt tarafta bir programcı `Application`'dan kendi somut `MyApplication` alt sınıfını oluşturur ve kendi türündeki belgelerle (`MyDocument`) çalışır. Soyut üst sınıftan gelen soyut metotların alt sınıflarda yeniden tanımlanması gerekir; bu yüzden framework o metodun var olduğunu bilir.

Çalışma biçimi şöyle öngörülmüştür. `Application` sınıfının `NewDocument()` metodu önce bir belge işaretçisi tanımlar ve `CreateDocument()` metodunu çağırarak belgenin oluşturulmasını sağlar. Bu işi asıl yapacak olan somut alt sınıftır, yani gerçeklenecek uygulamadır. Üst sınıf, kendi tanımladığı ve alt sınıfta mutlaka kodlanması gereken bu metot üzerinden, uygulamanın çalıştığı belge türüne uygun bir belgenin oluşturulmasını sağlar. Ardından belge işaretçisini açık belgeler listesine ekler ve belgenin kendini açmasını üst sınıfın soyut `Open()` metodu üzerinden sağlar; bu metodu da somut belge sınıfı gerçekler.

Mantık şudur: framework katmanında tanımlanmış soyut metotlar, yeni uygulamaların kendine özgü işlemleri yapabilmesi için bir fırsat verir. Her uygulamadan bağımsız olarak yapılabilecek işler ise somut metotlar üzerinden, kimi zaman uygulamadaki, kimi zaman ilgili belge türlerindeki somut metotlarla yürütülür.

3.1 Başka bir çözüm: generic sınıflar

Böyle bir gereksinim başka biçimlerde de kodlanabilir. Örneğin Java'da 1.5 sürümüyle gelen generic sınıf yapısı ya da C++'taki class template'ler kullanılabilir; C# da generic sınıfları Java'daki gibi destekler. Ancak bu çözümün zayıf tarafı dile özel olmasıdır: generic sınıf mantığı olmayan bir nesneye dayalı dille çalışıyorsanız bu çözümü kullanamazsınız. Tasarım kalıpları her nesneye dayalı dilde kullanılabilecek çözüm önerileri sunmaya çalışır. Hoca başka bir yöntem aklına gelmediğini, gelen olursa paylaşmasını istedi.

4. Factory Method'un Yapısı ve Uygulanabileceği Durumlar

4.1 Kalıp yapısı

Kalıpta bir oluşturucu sınıf ve ürünler vardır (ekrandan, 00:17:20):

```
Product (soyut)                 Creator (soyut)
   △                              FactoryMethod()  (soyut)
   │                              AnOperation()  ── ... product = FactoryMethod() ...
ConcreteProduct  <·····  ConcreteCreator
                              FactoryMethod() ── return new ConcreteProduct
```

Oluşturucu sınıfın bir ürün oluşturmaya yarayan metodu vardır: kalıba adını veren factory method. Bunun yanında bir ya da daha fazla somut operasyon bulunur; bu operasyonlarda bir ürün oluşturmak gerektiğinde factory method çağrılır. Oluşturucunun somut alt sınıfları ise ürüne özgü komutları çağırarak ürünü oluşturur.

Şemada `Creator` ile `Product` arasında bir ilişki çizilmemiştir, çünkü bu ilişki kalıp için zorunlu değildir. Hocaya göre yine de buraya kesikli bir ok çizilebilirdi: `AnOperation()` içinde `product = FactoryMethod()` dendiği için, üstteki soyut tür bile olsa `Product`'a bir bağımlılık (kullanma ilişkisi) vardır. Hoca bu oku ekranda çizmeye çalıştı ama kesikli çizgi çizemedi; okun kesikli olması gerektiğini vurguladı. Katılımcıların ayrıca açıklanmadığı, adlarından amaçlarının anlaşıldığı belirtildi.

Kalıp örneğinde bir framework kullanılmıştı, ama kalıp tanımında "oluşturucu ve ürünler" geçer. Yani kalıbın tek amacı framework'lerle çalışmak değildir; farklı türden ürünlerin oluşturulup kullanılması gereken senaryolarda da kullanılabilir. Bu yüzden kalıbın adı Factory Method'dur.

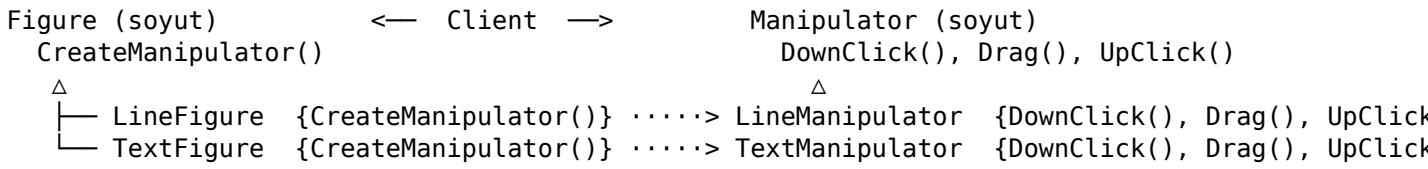
4.2 Uygulanabileceği anlar

Slayttaki maddeler (ekrandan, 00:20:00):

- Yeni nesneler oluşturması gereken bir sınıfın, oluşturacağı nesnenin türünü bilemediği yerlerde.
- Bir sınıfın kendi üzerindeki bir sorumluluğu çeşitli yardımcı alt sınıflarına ileteceği ve hangi tür yardımcı alt sınıfın oluşturulması gerektiği bilgisinin ise yerelleştirilmesi gerektiği yerlerde.
  - Örnek: Bir çizim uygulamasında şekiller fare ile değiştirilecek.

4.3 Figure ve Manipulator örneği

GoF kitabı bu ikinci durumu da bir örnekle verir (ekrandan, 00:22:40). Çeşitli grafik elemanları (figürler) ve onları değiştirmeye, düzenlemeye yarayan manipülatörler vardır.



Client figürler ve manipülatörler üzerinden çalışır. Örneğin kullanıcı bir çizgiye tıkladığında çizgiye özel bir değiştirici araç ortaya çıkar: ok ucu olup olmadığı, düz ya da kesikli olduğu, kalınlığı gibi ayarlar. Bir metne tıkladığında ise metni değiştiren bir araç çıkar: font, italik ya da kalın olup olmadığı, font boyu ve başka metin efektleri. Çizim programı ve client açısından ikisi de figürdür, ama düzenlenme biçimleri farklıdır.

Bunu modellerken "neyin değiştiğini bul ve sarmala" ilkesi uygulanır: figürler ve düzenleyiciler değişmektedir. Bu yüzden soyut bir `Figure` ve soyut bir `Manipulator` sınıfı kurulur. Bir `LineFigure` kendisinin `LineManipulator` tarafından, bir `TextFigure` kendisinin `TextManipulator` tarafından düzenleneceğini bilir.

Client bir figüre dokunup onu düzenlemek istediğinde aslında onun soyut `CreateManipulator()` metodunu çağırır. Figür bir `LineFigure` ise onun somut `CreateManipulator()` metodu çalışır ve client'a bir `LineManipulator` döner; metin için de aynısı geçerlidir. Böylece client, kendisine verilen manipülatörün özgül türünden bağımsız olarak, soyut üst `Manipulator` sınıfındaki soyut metotlar üzerinden somut alt sınıflara istediği işleri yaptırır. Tıklama, sürükleme, bırakma gibi ortak olaylar soyut üst sınıf üzerinden rahatlıkla yaptırılır.

Her figür türü için bire bir bir manipülatör olması da gerekmez. Bazı figürler özel bir değiştiriciye ihtiyaç duymaz; farklı figür türlerinin ortak kullanabileceği somut bir genel (generic) manipülatör sınıfı da olabilir. Yani ihtiyaçlar doğrultusunda her figür türü için ayrı bir manipülatör kodlamak gerekmeyebilir.

5. Uygulama Örneği: E-Ticaret Sitesi

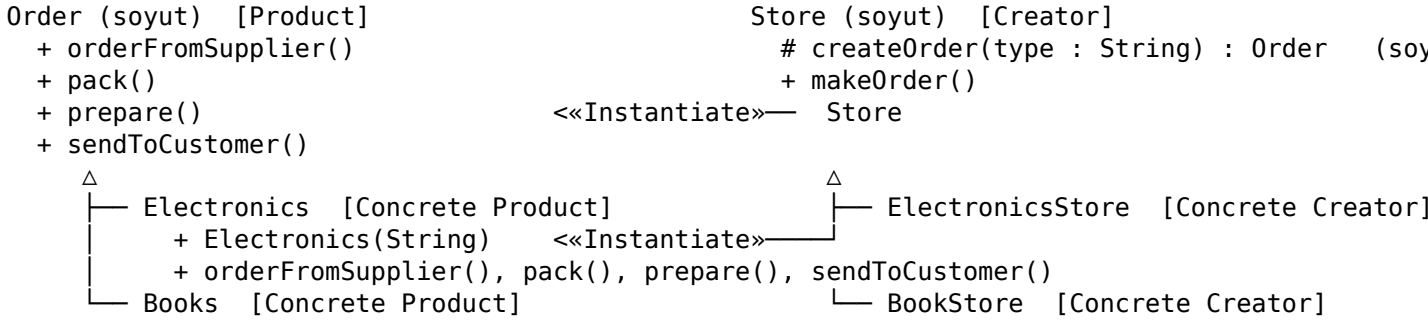
5.1 Problem

Hoca örneği Head First Design Patterns kitabından, birebir değil, bazı kısımlarını değiştirerek (esinlenerek) almış. Bir e-ticaret sitemiz var ve farklı türde ürünler satıyoruz: elektronik cihazlar, kırtasiye, giyim/tekstil vb. Müşteriler sipariş veriyor; site çalışanı siparişteki farklı türdeki ürünleri işleme koyuyor. Sipariş işleme adımları belirli bir mantık sırasıyla yürütülür, ama bu adımlardaki iş mantığı ürünün türüne göre farklıdır. Site çalışanına siparişleri tedarikçiye vermede kullanabileceği bir araç sağlamak istiyoruz.

Öngörülen kullanım şöyledir: elektronik ürün siparişi geldiyse çalışan bir elektronik mağazası (store) nesnesi oluşturur ve ürün koduyla sipariş çeker. Kitap siparişi geldiyse bir kitap mağazası oluşturur ve kitabın barkodunu, daha doğrusu ISBN'ini vererek siparişini oluşturur. Amaç, hangi tür sipariş gelirse gelsin çalışanın görece tek biçimli (uniform) bir yolla çalışabilmesidir.

5.2 Sınıf şeması

Sınıf şeması (ekrandan, 00:30:00):



```
+ Books(String)          <<Instantiate>>————————  BookStore
+ orderFromSupplier(), pack(), prepare(), sendToCustomer()
```

Hoca bu şemayı bir süre önce bir Eclipse eklentisiyle çizdiğini anlattı. Eklentinin avantajı kaynak kodu yazınca şemayı kendisinin çizmesi ve hangi ilişkilerin gösterilip gösterilmeyeceği gibi sadeleştirme komutları verilebilmesiydi. Dezavantajı ise Eclipse'e eklenti kurmanın kolay olmaması ve eklenti iyi desteklenmiyorsa Eclipse'in ya da Java'nın yeni sürümleriyle (örneğin JDK 1.5'ten 1.8'e geçişte) uyumsuz kalabilmesiydi. Bu yüzden bu tür şemalarda ufak tefek standart dışı şeyler olabilir.

Bunlardan biri `Store` ile `Order` arasındaki düz çizgidir. Düz çizgi sahiplik anlamına gelir, ama burada sahiplik yoktur: `Store` içinde yalnız soyut `Order` sınıfının somut alt sınıflarından birinin türünden bir nesne oluşturulur. Bu bir kullanma ilişkisidir ve şemadaki kahverengi «Instantiate» çizgilerinin kesikli çizilmesi gerekir.

Tasarım "neyin değiştiğini bul ve sarmala" ilkesine dayanır: farklı ürünler ve bu ürünlerin sipariş edileceği farklı mağazalar vardır, bu yüzden ikisi ayrı kalıtım hiyerarşileri içinde özelleştirilir. `ElectronicsStore` elektronik ürün nesneleri, `BookStore` kitap nesneleri oluşturup verir. Sipariş verildiğinde tedarikçiye sipariş vermek, ürün gelince paketlemek, faturasını ve başka belgelerini hazırlamak (örneğin elektronik üründe kutuya garanti belgesi koymak), sonra kargoyla müşteriye göndermek gibi adımlar her ürün için yapılır, ama farklı ürün türlerinde çok farklı biçimlerde yapılır. Bu yüzden bunları parametrelerle ayırmak yerine Factory Method kalıbı kullanılarak farklı türdeki ürünlerle çalışan farklı mağazalar oluşturulur.

Bu örnekte factory method `createOrder` metodudur. Public olan `makeOrder` metodunun içinde bu protected metot çağrılır. Hoca metodu bilerek `protected` yaptı: alt sınıflar bu metodu kodlayabilmeli ve çağırabilmeli, ama herhangi bir client onu kafasına göre çağıramamalıdır. Başka bir deyişle metot alt sınıflara public, diğer sınıflara private gibi görünmelidir; bunun yolu protected metot tanımlamaktır. `createOrder` hem protected hem de soyuttur.

5.3 Kod

Hoca örnekleri genelde iskelet kod olarak veriyor. `Order` soyut bir sınıftır ve bütün metotları da soyuttur. Hocaya göre tüm metotları soyut olan böyle sınıflar interface olarak da kodlanabilir, hatta hoca bunu tercih eder; ama örnekte soyut sınıf kullanılmıştır. `Electronics` sınıfında kurucu ürün kodunu (barkodu) alır; tedarikçiye siparişten müşteriye gönderime kadar yapılacak işlemler elektronik malzeme için nasıl yapılması gerekiyorsa o metotların gövdelerinde kodlanır. `Books` da benzer biçimde kodlanır, ama `orderFromSupplier`'dan `sendToCustomer`'a kadar olan gövdeler elektronik malzemeninkinden çok farklıdır.

`createOrder` metodunun parametresi slaytta önce `type` adını taşıyordu. Hoca derste bunu değiştirdi: parametre bir sınıf adı, tür adı ya da enum değeri değildir; kitap için barkod, elektronik için ürün kodu gibi bir benzersiz tanımlayıcıdır. Bu yüzden `uniqueID` adı daha uygundur. Düzeltilmiş kod (ekrandan, 00:37:40):

```
public abstract class Store {
    protected abstract Order createOrder( String uniqueID );
    public void makeOrder( String uniqueID ) {
        Order newOrder = createOrder( uniqueID );
        newOrder.prepare( );
        newOrder.orderFromSupplier( );
        newOrder.pack( );
        newOrder.sendToCustomer( );
    }
}
public class ElectronicsStore extends Store {
    protected Order createOrder( String uniqueID ) {
        Electronics newElectronicsOrder = new Electronics(uniqueID);
        //Elektronik malzeme siparişi için yapılacak işlemlerin tanımlanması
        return newElectronicsOrder;
    }
}
public class BookStore extends Store {
```

```
    protected Order createOrder(String uniqueID ) {
        Books newBookOrder = new Books(uniqueID);
        //Kitap siparişi için yapılacak işlemlerin tanımlanması
        return newBookOrder;
    }
}
```

`Store`'daki `makeOrder` yalnız `Order`'ın soyut metotları üzerinden çalışır; sipariş işleme adımlarının bu sırayla yapılmasını o belirler, ama her adımın nasıl yapılacağı somut sınıflarda kendilerine özgü biçimde belirlenir. `Store`'un alt sınıfları da `Order` hiyerarşisine paralel biçimde soyut `createOrder` metodunu somutlaştırır: hangi tür mağaza oluşturulduysa onun ürününü oluşturup geri döndürür. Arada başka işlemler de olabilir.

6. Kod Ayrıntıları: Annotation'lar ve Override

Hoca kodu Eclipse'te gösterirken (ekrandan, 00:40:00) annotation'lara değindi. `Electronics` sınıfının başı şöyledir:

```
package dp.factoryMethod.example;

public class Electronics extends Order {
    @SuppressWarnings("unused")
    private String barcode;

    public Electronics( String barcode ) {
        super( );
        this.barcode = barcode;
    }

    @Override
    public void orderFromSupplier() {
        //Elektronik malzeme için yapılacak işlemlerin tanımlanması
    }

    @Override
    public void pack() {
    ...
```

`@` ile başlayan satırlara annotation denir. Hocaya göre bunlar Java yorumlayıcısı için bir anlam ifade etmez; başka uygulamalara verilen sinyaller ya da bildirimlerdir. Örneğin Eclipse, JDK'yı anlık olarak çalıştırır ve hataları kenarda çarpıyla, uyarıları (warning) aşağıdaki listede gösterir. İskelet kod yazıldığı için `barcode` üye alanı sınıf içinde hiç kullanılmaz ve derleyici "tanımladığınız bir üye alanı hiç kullanmıyorsunuz" uyarısı verir. Bu uyarıyı görmemek için ya Eclipse ayarlarından (Preferences) o tür uyarı kapatılır ya da `@SuppressWarnings("unused")` yazılır. Bu annotation, derleyicinin döndürdüğü uyarı listesinden "kullanılmayan değişken ya da üye alan" türündekileri gösterme (bastır, suppress et) anlamına gelir.

`@Override` ise bir metodun, üst sınıftan kalıtımla gelen bir metodun alt sınıfta gövdesinin yeniden tanımlanmış hali olduğuna (overriding) dikkat çekmek için konur. Hocaya göre Java'da bunu koysanız da koymasanız da program aynı biçimde çalışır.

> **Ders sonrası not:** Hoca `@Override` için "Java için bir anlam ifade etmez" dedi. Programın çalışma anındaki davranışı açısından bu doğrudur. Ancak derleyici bu annotation'ı denetler: metot gerçekten bir üst sınıf (ya da arayüz) metodunu yeniden tanımlamıyorsa (örneğin metot adı yanlış yazılmışsa) derleme hatası verir. Hocanın kastı büyük olasılıkla çalışma zamanı davranışıdır.

Hoca .NET'te, özellikle C#'ta durumun farklı olduğunu anlattı. Normalde çok biçimlilik nedeniyle, üst sınıfın yeniden tanımlanmış (override edilmiş) bir metodu çağrıldığında alt sınıftaki hali çalışır. Hocanın anlattığına göre C#'ta bu davranış varsayılan olarak kapalıdır; override işareti konmazsa çok biçimlilik kullanılamaz. Hoca bunu yıllar önce C# ile nesneye yönelik programlama anlatırken, yazdığı bir kodun beklendiği gibi çalışmaması ve override eklenince düzelmesiyle fark ettiğini, durumun değişip değişmediğinden emin olmadığını söyledi.

> **Ders sonrası not:** C#'ta bu bir annotation değil `override` anahtar kelimesidir ve üst sınıftaki metodun da `virtual` (ya da `abstract`) tanımlanmış olması gerekir. Alt sınıfta `override` yazılmazsa alt sınıfın metodu üst sınıfınkini gizler (hiding; derleyici uyarı verir) ve nesneye üst sınıf türünden bir referansla erişildiğinde üst sınıfın metodu çalışır. Hocanın anlattığı davranış bu açıdan doğrudur; büyük olasılıkla güncel C# sürümlerinde de aynıdır.

Hoca kodda atladığı bir şey olmadığını kontrol ettikten sonra katılımcılardan bu kalıbı iş hayatında benzer amaçlarla kullananların deneyimlerini paylaşmalarını istedi.

7. Soru Üzerine: Kalıtım ve Çok Biçimlilik Nasıl Çalışır?

Aradan sonra bir öğrenci şu soruyu sordu:

Soru: Tepede soyut bir ana sınıf var, ondan gerçek (somut) sınıflar türetip metotları yeniden yazıyoruz. Ama bu alt sınıfların ayrımını soyut sınıf nerede yapıyor? Geçen derste bir switch-case yapısı vardı; burada geçiş nasıl sağlanıyor?

Cevap: Bu geçişi Java derleyicisi ve çalışma zamanında Java yorumlayıcısı kendisi halleder. Hoca bunu göstermek için Eclipse'te küçük bir örnek yazdı (ekrandan, 01:00:00):

- `Insan` adlı bir sınıf: bir isim üye alanı, isim alan bir public kurucu, isim için getter/setter ve `toString()` metodu. `toString()` aslında `Object` sınıfından kalıtımla gelir ve bir nesnenin kendisini tanıtan bir karakter katarı döndürmeye yarar; burada ismi döndürür.
- `Insan`'dan türeyen `Calisan` sınıfı: üst sınıfın kurucusunu kullanır ve bir firma bilgisi tutar. (Hoca normalde firmanın bir string ile değil, ad, adres gibi alanları olan bir `Firma` sınıfıyla modelleneceğini, örnek hızlı olsun diye string kullandığını belirtti.) `toString()` yeniden tanımlanır.
- `Insan`'dan türeyen `Ogrenci` sınıfı: firma yerine okul bilgisi tutar, `toString()` yeniden tanımlanır.

`toString()`'i yeniden tanımlarken iki seçenek vardır: üst sınıftaki `toString()`'in sonucunun sonuna ekleme yapmak ya da onu tamamen göz ardı edip kendi metnini hazırlamak. Hoca ikincisini seçti, çünkü üst sınıfınki "ben bir insanım" der; her çalışan bir insandır ama "ben bir çalışanım" demesi daha iyidir. Bu sırada isim alanına alt sınıftan doğrudan erişilemediği görüldü: üye alan veri gizleme ilkesi gereği `private` tanımlandığı için alt sınıf ona kalıtımla gelen getter metodu üzerinden erişir.

Deneme sınıfında `Insan` türünden bir dizi tanımlandı; elemanlarına birer `Calisan` ve `Ogrenci` nesnesi atandı ve dizi bir döngüyle (enhanced for) gezilerek her eleman ekrana yazdırıldı. Yazdırma sırasında otomatik olarak `toString()` metotları çalışır. Soru şudur: nesneler `Insan` dizisinde durduğuna göre kendilerini bir insan olarak mı, yoksa özelleşmiş halleriyle mi tanıtacaklar? Çıktıda (ekrandan, 01:40:00) her nesne kendi alt sınıfının metniyle tanıtıldı: çalışanlar "Ben … adlı şirkette çalışan …" ve öğrenci "Ben … adlı okulda öğrenci olan …" biçiminde. Bunu sağlayan çok biçimliliktir.

Hoca ardından `Insan` sınıfını soyut yaptı. Soyut sınıflara da kurucu tanımlanabilir, bir problem olmaz; yalnız artık deneme sınıfında doğrudan bir `Insan` nesnesi oluşturulamaz. Program yeniden çalıştırıldığında nesneler yine kendilerini özelleşmiş halleriyle tanıttı. Bunun nedeni, Java derleyicisinin soyut `Insan` sınıfını ve ondan türetilmiş somut sınıflar olduğunu bilmesi, bytecode'u buna göre oluşturması ve çok biçimliliği uygulamaya koymasıdır.

> **Hocanın vurgusu:** Bu, nesneye yönelimin en önemli kavramlarından biri olan kalıtımın en önemli yeteneklerinden biridir; bu yapı aklımızda olmalı.

8. Builder: Amaç, Örnek ve Çözüm

8.1 Amaç ve problem

Builder kalıbının amacı, karmaşık bir nesnenin oluşturma işlemleriyle gösterim işlemlerini birbirinden ayırmaktır. Böylece aynı oluşturma işlemi farklı gösterimler ortaya çıkarabilir.

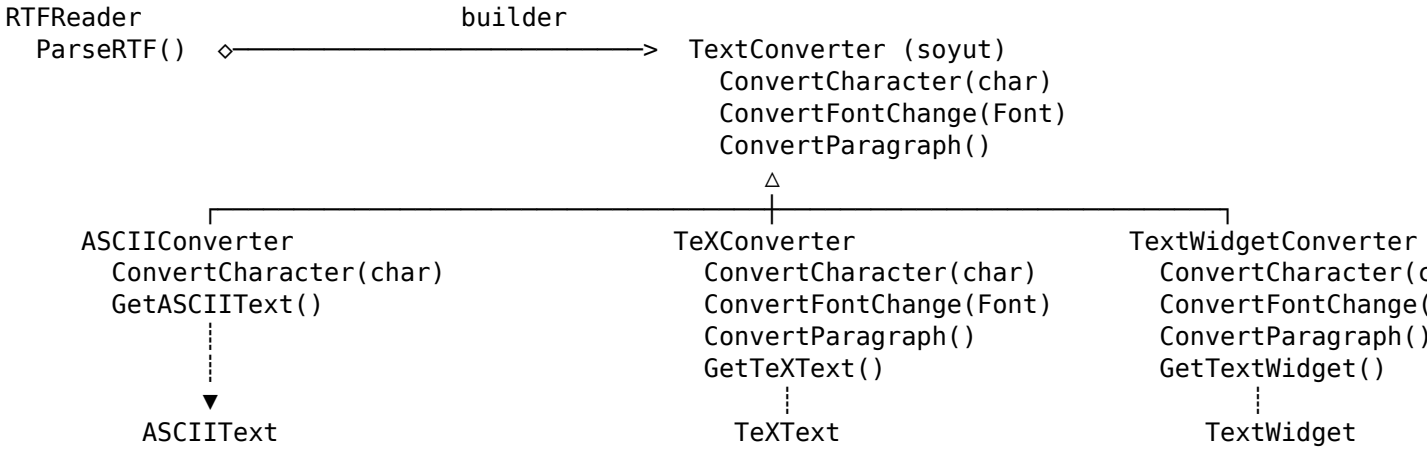
GoF'un örnek problemi şöyledir: bir Rich Text Format (RTF) okuyucu programı hazırlıyoruz. Program belgeyi başka metin türlerine çevirebilmeli, hatta o metni düzenleyebilecek bir GUI

bileşeni bile oluşturabilmelidir. Sorun şudur: Word'ün kullandığı format, PDF ve daha pek çok metin türü vardır. Hepsini ilk aşamada kodlamayacağız, zaman içinde yazılıma yeni özellik olarak ekleyeceğiz. Bu yüzden ileride yeni metin türü eklemenin zor olmamasını istiyoruz. Her şeyi bir arada yazarsak monolitik (yekpare) bir çözüm olur ve yeniliklere uyarlanması zorlaşır.

Önerilen çözüm: metni okumakla sorumlu bir sınıf, çeviri isteğini bir biçim çevirici nesneye göndersin. Çevirici nesne hem çevirme işleminden hem de çevirdiği metni belli bir biçimde temsil etmekten sorumlu olsun. Yeni metin türleri gerektiğinde bu çevirici sınıfın yeni alt sınıfları oluşturulur.

8.2 Çözüm şeması

Slayttaki şema (ekrandan, 01:10:00):



`ParseRTF()` metodunun notu:

```
while (t = get the next token) {
    switch t.Type {
    CHAR:
        builder->ConvertCharacter(t.Char)
    FONT:
        builder->ConvertFontChange(t.Font)
    PARA:
        builder->ConvertParagraph()
    }
}
```

Ana uygulama (RTF okuyucu) işi bir çevirici (dönüştürücü) üzerinden yaptırır. Ona bir ASCII dönüştürücü verilirse metni düz ASCII olarak gösterir ya da ASCII'ye çevirip bir dosya oluşturur; TeX dönüştürücü verilirse TeX (LaTeX) biçimine çevirir; TextWidget dönüştürücü verilirse fontu vb. düzenlenebilen bir GUI bileşenine çevirir. Okuyucu RTF dosyasını token token (bileşen bileşen) okur: yeni bir bileşen olduğu sürece onu alır ve türüne bakar. Karakterse çeviriciye "git bu karakteri çevir", font bileşeniyse "font değişikliğini ele al", paragraf bileşeniyse "paragrafı ele al" der. Başka token türleri varsa her biri için çeviricinin ilgili metodu çağrılır.

Bu ne kazandırdı? PDF'e çevirmek gerekirse `switch`'teki case'lere hiç dokunmadan yalnız `TextConverter`'ın yeni bir alt sınıfı olarak bir PDF dönüştürücü yazılır. Bu kolaylık, ürünün oluşturulmasıyla gösteriminin farklı sınıflarda ele alınması sayesinde sağlanır.

8.3 Çözümün eleştirileri

Slayttaki eleştiriler (ekrandan, 01:10:00):

- Çok biçimliliği tam kullanamadık, switch-case hâlâ duruyor.
- Alt sınıflar üst sınıfın tüm metotlarını gerçeklemelidir. Bu yüzden alt sınıfta anlamı olmayan metotlar boş gövdeli olarak kalmak zorunda.

Hoca birinci eleştiriyi çok da kusur saymadı: switch-case kalıbın çözmeye çalıştığı problem değildir, problemin doğasında vardır. RTF dosyası token token yazılır; okuyucu her token'ı okuyup türüne göre bir iş yapmak zorundadır ve bunun için başka şansı yoktur. Bu yüzden bu eleştiri "parantez içine" konuldu.

Asıl sorun ikincisidir. ASCII metinde font işlemi söz konusu değildir: her şey aynı düz biçimde yazılır ve metin okuyucu programda hangi font ayarlıysa bütün karakterler o fontla gösterilir. Yani `ConvertFontChange` metodu `ASCIIConverter` için anlamlı değildir. Ama üst sınıfta bu metot olduğu için alt sınıfta da mutlaka kodlanması gerekir; hiçbir şey yapmayan, boş süslü parantez gövdeli bir metot olur.

Hoca bunun kalıtımın temel mantığına uygun olmayan bir terslik olduğunu anlattı. Kalıtım, çok biçimlilik de düşünüldüğünde, alt sınıftan bir nesnenin üst sınıftan bir nesne beklenen her ortamda kullanılabilmesini, onun yerine geçebilmesini sağlar; çünkü alt sınıf üst sınıftaki bütün üyeleri almıştır. Bu üyeler değiştirilebilir ya da yeni üyeler eklenebilir, ama "şu üyeyi almıyorum" denemez.

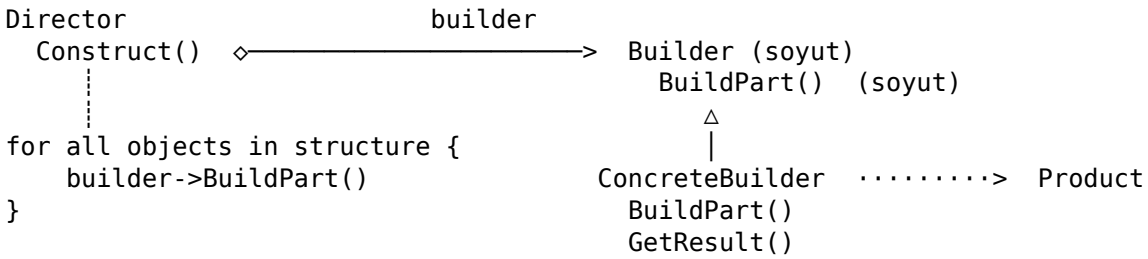
> **Hocanın vurgusu:** Hoca "inheritance" kelimesini bu yüzden "miras" değil "kalıtım" diye çevirir. Mirası reddedebilirsiniz, ama kalıtımla gelen genetik özelliklerinizi reddedemezsiniz; onlar size gömülüdür, yalnız değiştirebilirsiniz. Örneğin aileden gelen kıvırcık saçı düzleştirebilirsiniz ya da göz rengini lensle değiştirebilirsiniz, ama "bu geni almıyorum" diyemezsiniz.

Sonuçta hiçbir şey yapmayan boş gövdeli bir metot kodlanmış olur; bu bir kusurdur. Ama tasarımcının kusurundan çok, ASCII'de font diye bir şeyin olmamasından kaynaklanır.

9. Builder'ın Yapısı ve Factory Kalıplarından Farkı

9.1 Kalıp yapısı

Slayttaki yapı (ekrandan, 01:16:00):



Kalıp yapısında bir Director ve bir Builder bulunur. `Builder` sınıfı soyuttur ve oluşturulacak varlığın farklı parçalarını oluşturmak için farklı "build…" metotları vardır. Genelde tüm metotlarıyla birlikte soyut olduğu için interface olarak da kodlanabilir. Hangi somut tür ürün oluşturulmak isteniyorsa ona göre somut alt sınıflar yazılır. Director'da ürünün oluşturulmasını başlatan bir metot (`Construct()`) olur; hangi parçalar gerekiyorsa builder'ın ilgili metotları çağrılarak oluşturulur.

Slaytın altındaki not: kalıbın yukarıdaki tasarımında switch-case görünmüyor ama `ConcreteBuilder.buildPart` metodunda kendisini gösterecektir. Çünkü `Director.construct` metodunda "for all objects in structure" döngüsündeki nesneler bir şekilde `buildPart` metoduna parametre olarak aktarılmak zorunda kalınacak ve "parça şu ise şunu yap, bu ise bunu yap" gibi komutlar verilmek zorunda kalınacaktır. Hoca bunu da çok kusur saymadığını, asıl sorunun alt sınıflardaki gereksiz metotlar olduğunu yineledi.

Bileşenlerin görevleri:

- Builder: ürün nesnelerinin parçalarını oluşturmak için soyut metotlar üzerinden bir arayüz sağlar.
- Product: oluşturulacak üründür; içinde kendi parçalarıyla ilgili işlemleri yapan metotlar bulunur.
- Director: Builder arayüzünü kullanarak ürünü oluşturan sınıftır.
- ConcreteBuilder: Builder'ın bir gerçeklemesidir ve kardeş sınıfları olabilir. Ürünü parça parça oluşturur, birleştirir; bu işlerin nasıl yapılacağı onun içinde kodlanır. Ürünü teslim

etmek için `GetResult()` ya da `getProduct()` gibi bir metodu vardır.

Çalışma biçimi şöyledir: client (yazacağımız ana uygulama) önce oluşturulacak ürünün türüne ve sürümüne uygun bir somut builder oluşturur ve ilklendirir, örneğin az önceki örnekte bir PDF dönüştürücü. Sonra onu Director'a verir ve "hadi işe başla" der. Director'ın `construct` metodu çağrıldığında Director, duruma göre hangi parça türü gerekiyorsa onların somut builder üzerinden oluşturulmasını sağlar. En sonunda sonuç builder'dan alınır.

9.2 Uygulanabileceği anlar ve sonuçlar

Slayttaki maddeler (ekrandan, 01:20:00):

- Kalıbın uygulanabileceği anlar:
  - Karmaşık bir nesneyi oluşturma algoritmasının, nesneyi oluşturan parçalardan bağımsız olması gerektiği ve bu parçaların nasıl birleştirileceğinden bağımsız olması gerektiği anlarda.
  - Bir nesneyi oluşturma sürecinin bu nesnenin farklı gösterimlerinin olmasına izin verebilmesi gerektiği anlarda.
- Sonuçlar:
  - Ürünün iç yapısının en az yan etki ile değiştirilebilmesi mümkün olur (yeni bir ConcreteBuilder sınıfı yazarak).
  - Önceden gördüğümüz "factory" kalıpları ile karşılaştırıldığında, ürün oluşturma süreci üzerinde daha fazla denetim sahibi olunmakta (Director örneği Builder örneğine hangi işlemi hangi sırada yapması gerektiği emrini verir).

9.3 Builder ile Abstract Factory ve Factory Method'un farkı

Şimdiye kadar üç yaratımsal kalıp görüldü: Abstract Factory, Factory Method ve Builder. Builder'da ürünün oluşturulma süreci üzerinde daha fazla denetim vardır. Bu avantaj olarak da dezavantaj olarak da görülebilir: daha fazla denetim daha fazla bağımlılık gerektirir; öte yandan daha ince ayar yapmayı ve daha değişik işler yapabilmeyi sağlar.

Fark şudur: Builder'da Director nesnesi Builder nesnesine hangi işlemi hangi sırada yapması gerektiğini söyleyebilir. RTF örneğinde Director token'ları sırayla tek tek alıyordu; ama başka bir ürün türünde parçaları farklı sıralarla oluşturmak, bir parçayı hiç oluşturmamak ya da değiştirerek oluşturmak gibi komutlar verebilirdi. Builder'daki "build part" metotlarının hangi sırayla ve nasıl çalıştırılacağına Director karar verir.

Öteki kalıplarda ise durum farklıdır. Factory Method'daki mağaza örneğinde client yalnız "bana bir sipariş ver" der; nesnenin oluşturulması ve geri döndürülmesiyle ilgili bütün işleri ilgili somut factory sınıfı yapar ve nasıl yapacağına o karar verir, client karar vermez. Gerçi o örnekte client'ın böyle bir ihtiyacı da yoktu. Ama daha karmaşık bir ürün olsaydı ve client'ın oluşturma sürecinde daha fazla söz hakkına ihtiyacı olsaydı, ne Abstract Factory ne de Factory Method kullanılabilirdi. Factory Method'daki belge örneğinde de client "bana bir belge oluştur" der; belge oluşturulurken yalnız çok genel bazı adımlar üst soyut sınıfta kodlanır.

> **Hocanın vurgusu:** Bir kalıbın benzer kalıplarla farklarını kafanızda iyi oturtun. Ancak o durumda bir problemi en iyi hangi kalıpla çözebileceğinize karar verebilirsiniz. Ortada oluşturulacak bir ürün varsa diğer kalıplar elenir ve şimdiye kadar görülen üç kalıptan biri seçilir; doğru seçimi yapmak için bu üç kalıbın birbirinden farkını iyi bilmek gerekir. (Hocanın bu bağlamda verdiği uçak fabrikası sorusu "Örnek Sorular" bölümündedir.)

10. Uygulama Örneği: Araç Bayisi

10.1 Problem ve ürün yapısı

Bu örnek problemi hoca kendisi oluşturmuş (slaytta kaynak olarak hocanın baş harfleri yazılı). Slayttaki tanım (ekrandan, 01:30:00):

- Farklı marka ve model araçların satıldığı bir bayide markalara özel adlandırma ifadelerine rağmen ortak bir araç modellemesi yapılacak.

- Araçlar kasa, yakıt türü, motor hacmi ve gücü vb. bileşenlerin bir araya getirilmesi ile karmaşık bir süreçle oluşmaktadır.

Markalara özel adlandırma ifadelerinden kasıt şudur: örneğin Honda'nın araçlarına verdiği adlardan özellikleri anlaşılır. Model adında "back" (hocanın verdiği örnekte "Sportback" gibi) geçiyorsa araç hatchback'tir; "CRV" geçiyorsa SUV (jip) aracıdır; özel bir ifade geçmezse sedandır. Model adında "ECO" geçiyorsa LPG'ye dönüştürülmüş motorlu bir araçtır; "AT" geçiyorsa otomatik vitestir. Başka markaların kendilerine özel başka adlandırma yöntemleri vardır. Bu ifadeler aracın kasa tipini, yakıt türünü, motor hacmini ve beygir gücünü belirler; bunların bir araya getirilmesiyle karmaşık bir süreç ortaya çıkar.

Ürün `product` paketindedir (ekrandan, 01:30:00):

```
package product
  Car
    - brand, model : String
    ──> Transmission
          - gearCount : int {final}
          - isAutomatic : boolean {final}
    ──> Engine
          - volume, power : int
          ──> «enum» FuelType
                PETROLEUM, DIESEL, LPG, ELECTRIC
    ──> «enum» Chassis
          SEDAN, HATCHBACK, STATIONWAGON, SUV
```

Esas sınıf, üretilecek ürün `Car`'dır. Aracın vites bilgileri (kaç ileri vitesli olduğu ve otomatik olup olmadığı) `Transmission` sınıfındaki üye alanlarda tutulur. Motorun hacmi ve beygir gücü `Engine` sınıfında tutulur. Motor dört yakıt türünden birini kullanabilir: benzin, dizel, LPG ve elektrik; başka seçenek olmadığı için yakıt türü bir enum olarak tanımlanmıştır. Aracın kasası da sedan, hatchback, station wagon ya da SUV türlerinden yalnız biri olabileceği için enum'dur.

Hoca burada bir UML notu hatırlattı: `Engine`'den `FuelType`'a düz çizgili bir ok olduğuna göre aralarında sahiplik ilişkisi vardır, yani `Engine` sınıfında `FuelType` türünden bir üye alan olmalıdır. Şemada bu alan gösterilmemiştir, ama ok nedeniyle böyle bir alan olması gerektiği bilinmelidir. Benzer biçimde `Car` sınıfında `Engine` türünden bir üye alan olacaktır.

Sonuçta ürün, üye alanlarıyla birlikte epey farklı sayıda bileşen ve alt bileşenden oluşan karmaşık bir yapıdadır.

10.2 Kalıp seçimi

Parçaları karmaşık olan bir ürün var ve bunun farklı türlerini oluşturup müşterilere sunmak istiyoruz; bu yüzden hocanın aklına Builder geldi. Neden Abstract Factory değil? Çünkü "Honda ailesinden bir araç verdiğinizde vitesi hep şöyle, motoru hep şöyle olur" gibi bir durum yoktur; Honda'nın motoruyla ilgili farklı alternatif sürümler vardır. Abstract Factory'deki ürün ailelerinin aksine buradaki ürün kendi içinde çok daha fazla çeşitlenebilir. Factory Method ise bu ihtiyaç için basit kalır. Geriye Builder kalır.

10.3 Kod

`CarDirector` bir builder ile ilişkilidir ve builder'ı kurucusunda alır. Oluşturulacak aracın özelliklerini bir string dizisi olarak alır ve elemanları tek tek işler (hocaya göre marka/model ifadesi boşluklara göre token'lara ayrılarak da bu dizi elde edilebilir). En sonunda `report` metoduyla ne ürettiğinin bilgisini verir.

Soyut `CarBuilder` sınıfının altında bayinin sattığı her marka için somut bir alt sınıf vardır: `HondaBuilder` ve `RenaultBuilder`. Builder bir ürün (`Car`) oluşturur. Kurucunun yanında oluşturulan ürünü veren metot ve şasi, motor, vites oluşturan metotlar vardır; bütün build metotları kalıbın gerektirdiği üzere soyuttur. Kod (ekrandan, 01:34:00):

```
package dp.builder.example;
import dp.builder.product.*;
```

```
public abstract class CarBuilder {
    private Car product;
    public CarBuilder( String carModel ) {
        product = new Car(carModel);
    }
    public Car getResult() {
        if( product == null )
            return null;
        Car clone = new Car(product.getModel());
        clone.setBrand(product.getBrand());
        clone.setChassis(product.getChassis());
        clone.setEngine(product.getEngine());
        clone.setGear(product.getGear());
        return clone;
    }
    protected Car getProduct() { return product; }
    public abstract void buildChassis( );//Şasi'yi modele bağladım.
    public abstract void buildEngine(String token);
    public abstract void buildTransmission(String token);
}
```

Burada hem `protected getProduct()` hem de `public getResult()` vardır. `getProduct()` alt sınıfların kullanımı içindir: asıl ürüne erişim yetkisi alt sınıflara protected olarak verilmiştir. `getResult()` ise oluşan aracı Director'a vermek için gereklidir. Ürün oluşturulmamışsa `null` döner; oluşturulmuşsa hoca ürünün kendisini değil bir kopyasını döndürmeyi tercih etti. Hocaya göre bunun özel bir sebebi yok; asıl ürüne erişim protected olduğu için dışarıya kopyasını vermek zorunda hissetmiş.

`buildChassis()` parametre almaz, çünkü örneklere bakıldığında şasi bilgisi genelde model adından belirlenir. Motor ve vites için ise nasıl oluşturulacaklarının bilgisi parametre (token) olarak verilir.

Honda builder'ında marka "Honda" olarak işaretlenir. Şasi oluşturulurken model adında "back" geçiyorsa hatchback, "CRV" geçiyorsa SUV, ikisinin dışında bir şey geçiyorsa sedan kasa seçilir. Renault builder'ında da benzer kurallar vardır; hoca bütün modelleri araştırmadığını, her bileşen için bir iki örnek verdiğini söyledi. Örneğin adında "Captur" geçiyorsa SUV'dur; hiçbir şey belirtilmemişse varsayılan bir kasa türü atanır.

Honda'nın motor oluşturma metodu (ekrandan, 01:40:00):

```
public void buildEngine(String token) {
    //Min. values for a Honda engine
    if( getProduct().getEngine() == null )
        getProduct().setEngine( new Engine(FuelType.PETROLEUM, 1150, 60) );
    token = token.toUpperCase();
    if( token.contains("ECO") )
        getProduct().getEngine().setFuel(FuelType.LPG);
    else if( token.contains("PS") ) {
        int end = token.indexOf("PS");
        token = token.substring(0, end);
        getProduct().getEngine().setPower( Integer.parseInt(token) );
    }
    else if( token.contains("L") ) {
        int end = token.indexOf("L");
        token = token.substring(0, end);
        getProduct().getEngine().setVolume( (int)(Double.parseDouble(token) * 1000) );
    }
}
```

Henüz motor yoksa Honda için en düşük değerlerle (benzinli, 1150 cc, 60 beygir) bir motor oluşturulur; yani ek parametre verilmezse araç benzinli kabul edilir. Token'da "ECO" geçiyorsa yakıt LPG olur. "PS" geçiyorsa "PS" kısmı kesilir, kalan ifade sayıya çevrilir ve beygir gücü olarak atanır. "L" geçiyorsa bu motor hacmidir: "L" kesilir, kalan kısım sayıya çevrilir ve 1000

ile çarpılır. Hoca bunu şöyle açıkladı: ifadede örneğin 1.5 yazar; bu aslında 1500 cc demektir, motor hacmi cc olarak tutulduğu için 1000 ile çarpılır.

Vites için token'a bakılır: "AT" görülürse otomatik vites yapılır. Otomatikse ve vites sayısı belirtilmemişse 4, otomatik değilse ve vites sayısı belirtilmemişse 5 vites varsayılır.

Hoca bir önceki örnekteki switch-case'in burada bir if-else if zinciri olarak karşımıza çıktığını, çalışma mantığı olarak ikisinin aynı şey olduğunu belirtti. `Chassis` ve `FuelType` ilkel değil, sınıf olan (class) enum'lardır; Java'da enum'ların bu yeteneği için ilgili kitaplara bakılabilir.

10.4 Çalıştırma

`CarDirector`'ın `main` metodu ve çıktı (ekrandan, 01:42:10):

```
public void report( ) {
    System.out.println(builder.getResult());
}

public static void main(String[] args) {
    CarDirector director = new CarDirector( new HondaBuilder("Civic Sedan") );
    String elements[] = {"Engine\t125PS","Engine\t1.6L","Gear\t6AT"};
    director.construct(elements);
    director.report();
}
```

```
Car: Honda Civic Sedan (Sedan)
Engine: Petroleum 1600cc 125HP
Transmission: 6 gears  automatic transmission
```

Adım adım:

- Verilen: model "Civic Sedan"; ürün ayrıntılarından bilinen bilgiler 125 beygir, 1.6 litre motor, 6 ileri otomatik vites.
- `HondaBuilder("Civic Sedan")`: model adında "back" ya da "CRV" geçmediği için kasa sedan olur. Parantez içindeki "(Sedan)" kasanın türünü açıklayan, hocanın kodundan gelen açıklamadır.
- `"Engine\t125PS"`: motor yoktu, varsayılan motor (Petroleum, 1150, 60) oluşturulur; token "PS" içerdiği için "125" kesilip sayıya çevrilir, beygir gücü 125 olur.
- `"Engine\t1.6L"`: token "L" içerdiği için "1.6" alınır; $1.6 \times 1000 = 1600$, hacim 1600 cc olur.
- `"Gear\t6AT"`: "AT" otomatik vitesi, 6 vites sayısını verir.
- Yakıt türü hiç belirtilmediği için varsayılan benzin (Petroleum) kalır.

Hocaya göre Director parametreleri doğru yorumlayıp bir araya getirebildi.

11. Prototype: Amaç, Örnek ve Yapı

11.1 Amaç ve problem

Prototype kalıbının amacı, oluşturulacak nesnelerin türlerini belirlemek için prototip bir nesne kullanmak ve yeni nesneleri bu prototipi kopyalayarak oluşturmaktır.

GoF'un örneği: elimizde grafik uygulaması oluşturmaya yönelik hazır bir framework var. Bunu kullanarak yeni hobimiz olan müzik bestelemeye yarayacak bir program yazmak istiyoruz. Bunun yapılabileceğini düşünüyoruz, çünkü her nota aslında bir bitmap'tir. Porte üzerinde bir notanın hangi çizginin üzerinde ya da hangi iki çizginin arasında olduğu nota değerini belirtir. Tam nota, yarım nota gibi birkaç nota şekli vardır. Porte, sol anahtarı ve farklı nota türleri birer grafik elemanıdır; notanın porteye göre konumu da önemli bir bilgidir. Bunların hepsi bir grafik aracıyla oluşturulabilir.

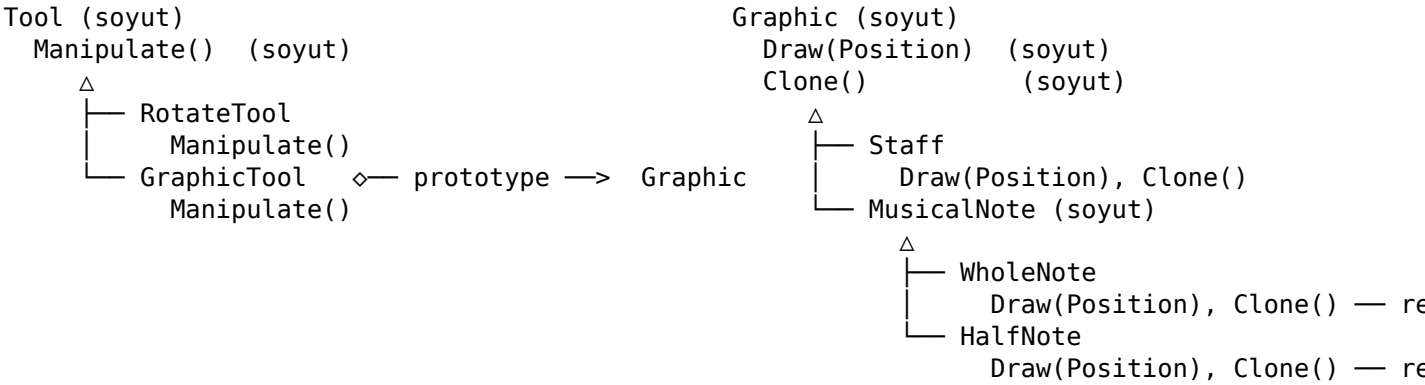
Bir resim programında daire, kare gibi şekilleri seçtiğimiz bir palet olduğu gibi, burada da paleti notalara uyarlarız: kullanıcı paletteki notaya tıklar, sonra o notanın resmini porte üzerinde bir yere sürükleyip bırakır. Böylece framework'teki grafik sınıfları ve paleti simgeleyen araç (tool) sınıfları kullanılabilir.

Sorun şudur: framework'ün farklı nota sınıfları hakkında önceden bilgisi olamaz. Grafik programı hazırlanırken bir müzik uygulaması olarak kullanılabileceği belki hiç düşünülmemiştir. O halde grafik aracı sınıfı, bilmediği sınıflardan çizim nesnelerini (notaları) nasıl oluşturabilir?

11.2 Çözüm

Grafik nesnesi kendisini herhangi bir konuma çizebilir ve kendisinin bir kopyasını oluşturmak için bir `Clone` metodu vardır. Fikir şudur: aynı tür notanın resmi değişmez; örneğin tam nota do da olsa re de olsa aynı resimdir, yalnız porte üzerindeki yeri değişir. Notalar arasında değişmeyen özellik çok olduğu için bir prototip nota oluşturulur ve her yeni nota için o prototip kopyalanır (klonlanır).

Slayttaki şema (ekrandan, 02:00:00):




```
GraphicTool.Manipulate():
    p = prototype->Clone()
    while (user drags mouse) {
        p->Draw(new position)
    }
    insert p into drawing
```

Önce porte (`Staff`) çizilir. Grafikte notalar da olabilir; tek başına "nota" denince hangi nota olduğu belli olmadığı için `MusicalNote` soyut yapılmıştır. Farklı nota türleri (tam nota, yarım nota vb.) `MusicalNote`'un alt sınıflarıdır; her birinin belli bir konuma çizilmesi ve kendi kopyasının oluşturulması için metotları vardır.

Hoca ekranda yine bir ayrım çizgisi çizdi: sol ve üst taraf framework, sağ alt çeyrek (`MusicalNote` ve alt sınıfları) bizim yazacağımız sınıflardır. Framework'te soyut bir `Tool` sınıfı vardır; hangi tür varlıkla ilişkili olduğuna göre farklı işlemler yapılabileceği için `Manipulate` metodu soyuttur. Somut araçlar örneğin döndürme aracı (`RotateTool`) ve bir grafik şekli oluşturmak için seçilen grafik aracıdır (`GraphicTool`). `GraphicTool` önce prototip nesneyi kopyalar, kullanıcı fareyi sürüklediği sürece resmi yeni konuma çizer, sürükleme bitince nesneyi çizime ekler.

> **Hocanın vurgusu:** Burada önemli bir püf noktası var: framework prototip yapısını kullanmamıza izin veriyor. Framework öngörülü davranmış, farklı alanlarda değişik uygulamalar oluşturulabileceğini düşünüp Prototype kalıbına destek sağlamış. Böyle çalışmasaydı bu framework'ten yola çıkarak müzik uygulaması yapılamazdı.

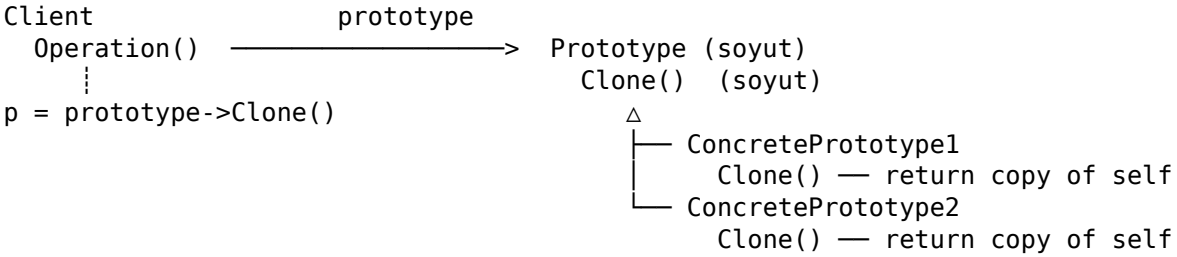
11.3 Factory kalıpları neden uygun değil?

Bu problem için Factory Method ya da Abstract Factory de ufak uyarlamalarla kullanılabilir, ama önemli bir dezavantajı vardır: her farklı nota için ayrı bir fabrika sınıfı gerekir. Üstelik ürün ailesinin ve ürünlerin ne olacağını belirlemek zordur: "tam nota ailesi, yarım nota ailesi" mi olacak, yoksa "do ailesi, re ailesi" mi? Her nota için ayrı fabrika sınıfı gerekecekse her nota ayrı bir ürün ailesi olur: do ailesi, re ailesi, mi ailesi; aile bireyleri de tam nota, yarım nota gibi çeşitlemelerdir. Yedi nota (do, re, mi, fa, sol, la, si) için yedi fabrika sınıfı gerekir; bunlara bir de ürün sınıfları eklenince sınıf sayısı çok artar. Hocaya göre problem bu kalıplara

düzgün oturmaz; oturtulmaya çalışılsa uzun uzun tartışılabilir, ama sorun çıkacağı şimdiden söylenebilir.

11.4 Kalıp yapısı ve bileşenleri

Slayttaki yapı (ekrandan, 02:05:10):



- Prototype: nesnelerin kendisini klonlaması için bir arayüz sunar.
- ConcretePrototype: kullanılacak nesnelerdir.
- Client: bir prototipin kendisini kopyalamasını isteyerek yeni bir nesne oluşturur.

Client her zaman bir prototip üzerinden çalışır: bir işlem yapmadan önce prototip nesnenin bir klonunu oluşturur ve işlemlerini bu yeni kopya üzerinde yürütür. Bu yüzden `Prototype` soyut bir `Clone` metoduna sahip olmak zorundadır ve her somut prototipin kendini klonlayan bir `Clone` metodu olmalıdır.

11.5 Uygulanabileceği anlar

Slayttaki maddeler (ekrandan, 02:10:00):

- Kalıbın uygulanabileceği anlar:
  - Bir sistemin, kendisine ait ürünlerin nasıl oluşturulduğu, birleştirildiği ve sunulduğundan bağımsız olmasının istendiği durumlar
  - ve şu durumlardan biri söz konusu ise (Factory Method kalıbından farklılaşmayı sağlıyor):
    * Oluşturulacak sınıflar çalışma anında belirlenecekse
    * Ürünlerin sınıf hiyerarşisi ile paralel bir factory hiyerarşisi kurmak istenmiyorsa
    * Ürün sınıfının örneklerinin durum uzayı sınırlı ise (bu durumda sınıfın iyi tasarlanıp tasarlanmadığı ayrıca tartışılır!)
- Bu kalıp C++ gibi sınıfların doğrudan işlenebilecek varlıklar olmadığı dillerde daha çok yarar sağlar.
  - SmallTalk gibi geç ilişkilendirme (late binding) ve Java gibi yansıtma (reflection) yetenekleri olan dillerde bu kalıp yerine dilin yetenekleri kullanılabilir.

İlk madde şöyle açıklandı: bize bir prototip verildikten sonra yalnız "kopyala" deriz, klonunu oluştururuz, sonra bir iki niteliğini değiştirerek istediğimiz yeni varlığı elde ederiz. Prototipin ilk oluşturulma aşamasının hangi adımlardan geçtiği bizi ilgilendirmez; bize şeffaftır. Ancak bu madde Factory Method ve Abstract Factory için de söylenebilir; bu yüzden üç alt durumdan en az birinin de geçerli olması gerekir.

- Oluşturulacak sınıfların çalışma anında belirlenmesine örnek: bir oyunda savaşçı ve büyücü karakter sınıfları var; sonradan bu iki sınıfın bazı niteliklerini içeren bir suikastçı sınıfı oluşturmak istiyoruz ve oyuncunun sınıflar arasında geçiş yapabilmesini istiyoruz.
- Paralel factory hiyerarşisi istenmemesine örnek nota örneğidir: birden fazla fabrika, hatta aralarında kalıtım ilişkisi olan bir fabrika hiyerarşisi gerekecekse ve bundan kaçınmak isteniyorsa fabrikalar yerine Prototype kullanılır.
- Üçüncü madde için hoca, anlatım sırasında bununla tam olarak ne kastettiğini hatırlamadığını söyledi; bu madde dersin sonunda bir öğrencinin sorusuyla yeniden tartışıldı (bkz. "Örnek Sorular").

  Ders sonrası not: GoF kitabında bu madde büyük olasılıkla şu anlamdadır: bir sınıfın örnekleri yalnız birkaç farklı durum birleşiminden birine sahip olabiliyorsa, her birleşim için bir prototip kurup gerektiğinde klonlamak, sınıfı her seferinde uygun

durumla elle oluşturmaktan daha kolaydır. Yani slayttaki “durum uzayı sınırlı ise” ifadesi doğrudur.

Dillerle ilgili ipucu şöyle açıklandı: C++’ta sınıflar doğrudan birinci sınıf programlama varlıkları (first-class entity) gibi kullanılamaz; template yeteneği vardır ama bu ayrı bir üst kümedir. Var olan sınıfları ve türleri çalışma anında değiştirmek egzotik bir işlemdir. Java’nın reflection yetenekleri vardır: bir nesne için `getClass()` dendiğinde o nesnenin sınıfını simgeleyen bir nesne elde edilir ve onun üzerinde çok değişik şeyler yapılabilir; hocaya göre çalışma anında bytecode üzerinde işlem yaparak bir sınıfa yeni üye alan ya da metot eklemek, var olan bir metodun gövdesini değiştirmek bile mümkündür. Bu özelliklerle Prototype’tan daha iyi bir çözüm elde edilebilir, ama o çözüm dile özel olur: o altyapı başka dillerde yoktur ya da farklı biçimlerde vardır, kod başka platforma taşınamaz.

Smalltalk ilk nesneye dayalı dillerden biridir ve çok değişik özellikleri vardır; örneğin tiplemesi daha esnektir: bir nesne işaretçisine nesne atarken işaretçinin türünü önceden belirlemek zorunda değilsiniz. Ama bu özellikler sektörde ana akım olarak yaygınlaşmadı. Bu yüzden Prototype gibi her platformda uygulanabilecek çözümler kurulur.

> **Hocanın vurgusu:** Ders notlarındaki örnekler Java ile yazılmıştır, ama neredeyse tamamı C#’a kolayca çevrilebilir (yalnız package yerine namespace denir); C++’a da gösterimi biraz daha karmaşık olsa da rahatça çevrilebilir. Dile ya da platforma özel durumlara yalnız değiniyoruz.

12. Sığ Kopya ve Derin Kopya

Prototip kurarken klonlama yapılacağı için şu soru doğar: kopyalama sığ mı (shallow copy) olacak, derin mi (deep copy)? Kopyalanacak nesnenin ilkel olmayan, başka sınıflardan nesneler olan üye alanları varsa fark ortaya çıkar. Nesne kopyalanırken ilkel olmayan üye alanların da kopyaları (klonları) oluşturuluyorsa buna deep copy denir. Yalnız nesnenin kendisi kopyalanıp üye alanlardaki nesneler kopyalanmıyorsa shallow copy denir.

> **Ders sonrası not:** Hoca sığ kopyada ilkel olmayan üye alanların “null olarak kopyalandığını” söyledi. Doğrusu: sığ kopyada bu alanlardaki referanslar kopyalanır; kopya ile asıl nesne aynı alt nesneleri paylaşır (Java’da `Object.clone()` varsayılan olarak böyle çalışır). Bu alanlar null olmaz; sorun, birinde yapılan değişikliğin ötekini de etkilemesidir.

İkisinin de avantajı ve dezavantajı vardır. Sığ kopya, paylaşılan referanslar yüzünden yetersiz kalabilir. Derin kopya ise döngüsel bir ilişki varsa sorun çıkarır. Hoca bunu bir örnekle anlattı:

- Bir `Insan` sınıfı olsun. Her insanın en az iki ebeveyni vardır: anne ve baba (birer `Insan`). Vefat etmiş bile olsa kayıtlarda annesi ve babası vardır.
- Bir insanın çocukları da olabilir: bir `Insan` dizisi, niceliği `0..*`. Yani bir insan nesnesi en az iki, belki daha fazla insan nesnesiyle ilişkilidir.
- Bir insanın klonunu oluştururken annesini ve babasını simgeleyen nesnelerin de klonlarını oluşturup mu vereceğiz? Öyleyse deep copy, yalnız insanı kopyalıyorsak shallow copy. Hocaya göre shallow copy yapılırsa aradaki ebeveynlik ilişkileri bozulur.
- Deep copy yapalım: “beni” kopyalarken annemin de klonunu oluşturuyoruz. Annemin klonunu oluştururken onun çocukları arasında yine ben varım; benim bir klonum daha oluşuyor. Babamın klonunu oluştururken de aynısı oluyor.
- Şimdi ilk oluşturulan klonum, annemin çocukları arasındaki klonum ve babamın çocukları arasındaki klonum aynı nesne mi? Olmayabilir. Olmazsa garip bir problem çıkar: bir kopya isterken benden üç kopya olur.

Bu yüzden derin kopya yapıldığında `clone` metodu karmaşıklaşabilir; döngüselliği engellemek için gerekli önlemleri almak gerekir. Bu beklendiği kadar kolay değildir; gerçek bir sistemde epey karmaşık noktalara varılabilir.

Hoca bir ayrıntı daha ekledi: kalıp şemalarında gösterilmese de klonların üye alanları olacak ve bu alanlara değer atamak, değerleri öğrenmek için en azından ilgili getter ve setter metotları gerekecektir.

13. Uygulama Örneği: Oyun İçin Level Editörü

13.1 Problem

Örnek Head First Design Patterns kitabından alınmıştır (ekrandan, 02:20:00):

- Bir bilgisayar oyunu için level editörü hazırlanıyor.
- Haritaya çok çeşitli türlerden canavarları (örneğin sürükle-bırak ile) kolayca ekleyebilmek istiyoruz.
- Ama bir canavar oluşturmak kolay iş değil, bir sürü durum bilgisi var: HP, DEX, AGL, LUCK, CHR vb.

HP (hit point) sağlığı, DEX (dexterity) el çabukluğunu, AGL (agility) çevikliği (saldırılardan ne kadar hızlı kaçınabildiğini ya da ne kadar hızlı hareket edebildiğini), LUCK şansı, CHR (charisma) diğerlerini etkileme gücünü simgeler; başka nitelikler de olabilir. Basit canavarların yanında örneğin bölüm sonuna ya da kritik bir darboğaza güçlü bir canavar konmak istenebilir; güçlü olması için örneğin HP'si yüksek olacaktır. Belli değer aralıklarına sahip canavarlara bir isim verilip paletten kolayca seçilebilmesi istenir.

13.2 Çözüm şeması

```
«Abstract» Monster                          [Prototype]
  # HP, DEX, AGL, LUCK, CHR : int
  # type : String
  + clone() : Monster                       (abstract)
  + getType() : String
  # Monster(int, int, int, int, int)
     △
GenericMonster                              [Concrete Prototype]
  + clone() : Monster
  + GenericMonster(int, int, int, int, int)

MonsterRegistry  1 ──────> *  GenericMonster
  + findMonster( type : String ) : Monster
  + registerMonster( Monster )

MainApp  [Client]  ··> MonsterRegistry
MainApp            ··> GenericMonster
```

(ekrandan, 02:20:00)

Genel bir `Monster` sınıfında her canavarın sahip olması gereken üye alanlar vardır; türlere isim vermek için bir `type` alanı da vardır. Kurucu tüm üye alanları ilklendirir; tür adı için bir getter vardır; bir de Prototype kalıbının olmazsa olmazı `clone` metodu vardır. Hoca hangi öğelerin soyut olduğunu açıkladı: kurucu soyut olmaz; getter basit bir metottur, o da somuttur. Bu sınıfta yalnız sınıfın kendisi ve `clone` metodu soyuttur. Kalıp açısından `Monster` Prototype, `GenericMonster` ise Concrete Prototype rolündedir. Kod anlatılırken `Monster`'daki kurucu, getter ve setter metotlarının normal somut metotlar olduğu yinelendi.

`GenericMonster` sahip olması gereken üye alanları kurucuda alır ve türünü "genel canavar" olarak belirler. Hoca kendi özel canavarlarını oluşturdukça türünü değiştirip ona göre işlem yapacağını anlattı. `clone` metodu, kendi durum bilgisiyle aynı durum bilgisine sahip yeni bir nesne döndürür. Burada sığ ya da derin kopya ayrımına gerek yoktur, çünkü bütün üye alanlar ilkeldir.

Canavarları her seferinde üye alanlarını yeniden ayarlamadan getirebilmek için bir canavar kayıt defteri (`MonsterRegistry`) vardır. Bu sınıf canavarları bir map (tablo) içinde tutar: anahtar canavarın türü olan karakter katarı, değer canavarın kendisidir. Kurucuda bu tablo ilklendirilir; belli bir tür canavarı tabloda arayıp bulan ve belli bir canavarı kaydeden metotlar vardır. Ana uygulama (level editörü) kalıptaki client'tır: kayıttan bir canavar ister; ilgili canavar bulunamazsa mevcut canavarlardan birinin klonu alınıp yeni bir canavar türü oluşturulur.

13.3 Ana uygulama

Kod (ekrandan, 02:23:30):

```
public class MainApp {
    public static void main(String[] args) {
        MonsterRegistry reg = new MonsterRegistry();
        reg.registerMonster( new GenericMonster(10, 10, 10, 10, 10) );
        Monster mySpecialMonster =
            reg.findMonster("Generic Monster").clone();
        mySpecialMonster.setHP(100);
        mySpecialMonster.setType("Bolum sonu canavarı");
        reg.registerMonster(mySpecialMonster);
    }
}
```

Önce bir kayıt defteri oluşturulur ve tüm nitelikleri 10 olan basit, düşük güçlü bir genel canavar kaydedilir. Yeni bir canavar oluşturmak için kayıttan genel canavar istenir ve klonlanır; böylece yeni canavar nesnesi elde edilir. Onun HP'si epey yüksek bir değere (100) ayarlanır, türü "Bölüm sonu canavarı" yapılır ve o da kayda eklenir.

14. Singleton

14.1 Amaç ve örnekler

Singleton kalıbının amacı, bir sınıftan yalnız bir nesnenin oluşturulabilmesi, başkasının oluşturulamamasıdır. Hoca bunun çok karmaşık olmayan, kısa bir kalıp olduğunu söyledi.

Ne zaman gerekir? Örneğin bir bilgisayara istediğiniz kadar yazıcı takabilirsiniz, ama yazdırma biriktiricisi (print spooler) süreci bir tanedir. Hangi yazıcıya yazdırırsanız yazdırın önce print spooler devreye girer, işi alır ve ilgili (varsayılan) yazıcıya yönlendirir. Benzer biçimde çekirdek süreci tektir; başka kopyalarının oluşturulması istenmez, oluşturulamamalıdır. Tek bir dosya sistemi yönetimi (aynı disk bölümünün yarısı bir dosya sistemiyle, yarısı başka biriyle yönetilemez) ve aynı anda tek bir pencere yöneticisi de örnektir. Kısacası belli bir sınıftan yalnız tek bir nesne kullanılsın ve o türden nesne isteyen herkese o nesne verilsin, başka bir nesne verilmesin isteniyorsa bu nasıl kodlanır?

14.2 İlk akla gelen: statik üye alan

İlk akla gelen, o nesneyi statik bir üye alan olarak tanımlamaktır; örneğin bir sınıfta `PrintSpooler` türünden statik bir üye alan. Ama bu, nesnenin yalnız o sınıf içinde ortak kullanımını sağlar. Nesneye farklı türden nesneler erişmek isteyecekse bu ortak alanı hangi sınıfta tanımlayacağız? Statik alana sınıf üzerinden erişilir, ama doğru sınıfa mı koyduk, başka bir sınıfa koysak daha mı iyi olurdu? Hoca bu sorunun biraz akademik kaçabileceğini söyledi. Ayrıca statik üye alan tanımlandığında derleyicinin oluşturduğu o tek statik kopya üzerinde daha fazla söz söyleme şansınız olmaz, ona çok müdahale edemezsiniz. Nesneyi daha esnek kodlamak ve kullanmak istiyorsanız Singleton kalıbı uygulanır.

14.3 Önerilen çözüm

Kalıp az önceki soruya da yanıt verir: ortak kullanılacak nesne hiçbir başka sınıfa atanmasın, kendi sınıfı tarafından yönetilsin. Böylece o tek nesne üzerinde, sınıfın kodu üzerinden daha fazla söz hakkımız olur. Slayt (ekrandan, 02:40:00):

- Önerilen çözüm: Bir sınıfa, kendisinin tek bir örneğinin olmasını sağlama sorumluluğunu atamak.
  - Sınıfın kendi türünden static bir üyesi olur,
  - Sınıfın kurucusu private tanımlanarak kurucuya erişim engellenir,
  - Sınıfın bu üye için bir factory metodu bulunur.
- Kalıba adını veren bu sınıfın adı Singleton olarak belirlenmiştir.

Hoca sorumluluğun bu sınıfa verilmesinin uyumu (cohesion) da yükselttiğini belirtti.

```
Singleton
  - uniqueInstance : Singleton {static}
  - state : SingletonData
  - Singleton( ... )
  + getInstance( ) : Singleton {static}
  + singletonOperation( )
  + getterMethods( )

getInstance():
    if( uniqueInstance == null ) {
        uniqueInstance = new Singleton( ... )
    }
    return uniqueInstance;
```

`uniqueInstance` ortak kullanılacak nesnedir ve bu sınıftan oluşturulabilecek tek nesnedir; bu yüzden statiktir. Singleton'ın çeşitli üye alanları olacaktır; bunları simgelemek için şemada `SingletonData` türünden bir `state` alanı konmuştur (farklı türlerden farklı üye alanlar olabilir). Kurucu `private` olduğu için herkes çağıramaz, yalnız sınıfın kendisi çağırabilir. Tek örneği döndüren `getInstance()` metodu da statik olmalıdır, çünkü döndürdüğü nesne statiktir ve metot nesne oluşturulmadan çağrılabilmelidir. Metot önce `uniqueInstance`'ın daha önce ilklendirilip ilklendirilmediğine bakar; ilklendirilmemişse ilklendirir, sonra onu döndürür. Daha önce ilklendirilmişse son hali neyse o döner.

14.4 Avantajları ve ne zaman kullanılır?

Bu altyapı sınıftan tek bir nesne olmasını sağlar. Singleton'ın üzerine yararlı operasyon metotları eklenebilir ve böylece nesnenin davranışı üzerinde daha fazla söz hakkı olur; yalnız statik bir nesne tanımlamak bir anlamda Singleton'ı kullanmak gibidir, ama o nesne üzerinde çok söz hakkı bırakmaz. Ayrıca Singleton sınıfından kalıtımla, yine tek olması gereken başka singleton sınıfları kolayca türetilebilir. Statik alan ihtiyaç duyan sınıflardan birine konsaydı, oradan kalıtımla yeni singleton'lar türetmek mümkün olmazdı; Singleton başlı başına bir sınıf olduğu için bu kolaydır.

Singleton'ın tek bir kullanıcısı olacaksa statik bir alan yeter, kalıbı kurmaya gerek kalmaz. Ama farklı türlerden kullanıcılar olacaksa Singleton kalıbı kurulmalıdır. Kısacası bazı nesnelerin türünün tek örneği olması ve bu örneğin farklı türden nesneler tarafından kullanılması isteniyorsa tercih edilecek kalıp Singleton'dır.

> **Hocanın vurgusu:** İleride göreceğimiz Strategy ve State kalıplarının gerçeklemeleri iyi birer Singleton adayıdır; bu kalıplar anlatılırken bu nokta yeniden hatırlatılacak.

Tek sınıftan oluşan bir kalıp olduğu için hoca bunun ayrıca bir örnek gerçeklemesini yapmadı. Böylece yaratımsal kalıplar (Abstract Factory, Factory Method, Builder, Prototype, Singleton) tamamlanmış oldu.

Örnek Sorular

Soru 1: Silah ve şarjör sınıflarının UML şeması (ekrandan, 02:30:00)

Hocanın sayfasında paylaştığı, notlandırılmayan bir kendini sınama alıştırmasının (quiz) beşinci sorusu ekranda gösterildi: "Yukarıdaki sorularda kodladığınız sınıfların tümünü içeren UML sınıf şemasını çiziniz." Hoca, alıştırmanın tasarımı ana hatlarıyla verip ayrıntısını doldurtan değil, ihtiyaçları verip kendi tasarımınızı yapmaya yönlendiren bir soru olduğunu söyledi: farklı ilişki türleri seçilebilir, ama her seçimin bu problem için avantajları ve dezavantajları vardır. Önceki soruların metni ekranda görünmedi.

Hocanın çözüm şeması:

```
Şarjör
  - mermiKapasitesi, mevcutMermi : int
  + Şarjör(mermiKapasitesi, mevcutMermi : int)
  + atesEt( ) : boolean
```

```
  + doldur( )
  + getMermiKapasitesi( ) : int

Silah
  - menzil : int
  - sarjor : Şarjör
  + Silah( int )
  + atesEt( hedefinUzakligi : int ) : boolean
  + setSarjor( Şarjör )
  + doldur( )

Silah  1 ──────> 0..1  Şarjör          (sahiplik, düz ok)

OtomatikSilah  ──▷  Silah              (kalıtım)
  - seriAtim : final int
  + OtomatikSilah( menzil, seriAtim : int )
  + atesEt( int ) : boolean

MainProg  ··>  Şarjör                  (kullanma, kesikli ok)
MainProg  ··>  OtomatikSilah           (kullanma, kesikli ok)
```

Soru (bir öğrenci): Main program ile diğer sınıflar arasında neden bağımlılık (dependency) ilişkisi var?

Cevap: Kesikli çizgi iki durumdan birinde konur. Birincisi: arada sahiplik ilişkisi yoktur, ama bir metodun içinde o sınıftan nesneler oluşturulur ve kullanılır. `MainProg` böyledir: üye alan olarak şarjör ya da otomatik silah tanımlanmaz; `main` metodunun içinde bir şarjör ve bir silah oluşturulur, silaha şarjör takılır vb. Bu yüzden sahiplik (düz çizgi) değil, kesikli çizgiyle kullanma ilişkisi gösterilir. İkincisi: üye alan yoktur, ama o sınıf bir metodun parametresidir; örneğin `MainProg`'da `init` diye bir metoda parametre olarak bir silah nesnesi verilirse `MainProg` ile `Silah` arasına ucu açık oklu kesikli bir çizgi çizilir. Kesikli çizgi bağımlılığı da gösterir: `MainProg`'u başka bir projeye kopyalamak isterseniz `Şarjör` sınıfını da kopyalamanız gerekir; sahiplik olmasa da `MainProg` `Şarjör` türüne bağımlıdır. Hoca UML ilişkileri için Nesneye Yönelik Programlama ders notlarına bakılmasını önerdi.

Soru 2: Hangi yaratımsal kalıp? (derste sözlü, 01:23:53)

Hoca kalıpların farklarını bilmenin önemini anlatırken şöyle bir soru tipi verdi: Bir uçak fabrikasında uçak üretilecek. Fabrikada hem savaş uçakları hem kargo uçakları üretiliyor. Aynı savaş uçağının farklı bileşenleri olabiliyor; örneğin farklı motor türleri (hocanın saydıkları altyazıda "After Jet Motor, Turbo Motor, Turbo Block Motor" olarak geçiyor). Bunların değiştirilmesine yönelik bir kod yazılacak. Abstract Factory, Factory Method ve Builder'dan hangisi kullanılmalı?

> **Çözüm (derste verilmedi):** Derste anlatılan ölçütlere göre karar şöyle verilebilir. Bir ürün çeşidinin bileşenleri hep birlikte sabit bir aile olarak geliyorsa (ör. "savaş uçağı ailesi" hep aynı türden motor, kanat vb. ile) Abstract Factory uygundur. Ürün tek parça oluşturuluyor ve hangi somut türün oluşturulacağına alt sınıf karar veriyorsa Factory Method yeter. Soruda olduğu gibi aynı uçağın bileşenleri (motor türü vb.) birbirinden bağımsız olarak çeşitleniyor ve oluşturma süreci parça parça yönetilmek isteniyorsa, araç bayisi örneğindeki gerekçeyle (bir marka "ailesi" sabit bileşenlerle gelmiyordu) büyük olasılıkla Builder en uygun seçimdir. Kesin karar sorunun tam ifadesine bağlıdır.

Soru 3: Araç örneği Prototype ile çözülemez mi? (derste sözlü, 02:25:00)

Soru (bir öğrenci): Builder'daki araba örneğini Prototype ile gerçekleştirmek daha mantıklı olmaz mı? Birden fazla yeni araç üretilebilir ve bunlar bir HashMap'te tutularak daha genel (generic) biçimde araba üretimi sağlanamaz mı?

Cevap: Gerçek dünya problem alanına bakılırsa bir araba modeli o kadar çok çeşitlendirilmez. Oyun örneğinde beş farklı nitelik vardır; bunlar çok daha fazla olabilir ve her biri çok farklı

oranlarda, farklı değer aralıklarında değişebilir. Arabada ise bileşenlerin nitelikleri üç aşağı beş yukarı daha sınırlı bir durum uzayında değişir. Hoca bu nedenle Prototype'ın araba için uygun olmayacağını düşündüğünü söyledi.

Ardından başka bir öğrenci, Prototype'ın uygulanabileceği durumlardaki üçüncü maddenin durum uzayıyla ilgili olduğunu, Builder ile Prototype'ı ayıran şeyin "sınırsız" bir durum uzayı olup olmadığını sordu; hoca önce bunu onayladı. Hemen sonra kendi söylediğinin slayttaki maddenin tersi olduğunu fark etti: slayt "durum uzayı sınırlı ise" diyordu ve araba için durum uzayı daha sınırlıdır. Hoca, slayttaki ölçütün Prototype'ı Builder ile değil factory kalıplarıyla karşılaştırdığını belirtti.

> **Ders sonrası not:** Bu tartışmada iki ayrı karşılaştırma karışmıştır. Slayttaki madde (ve büyük olasılıkla GoF'taki karşılığı) Prototype'ı factory kalıplarına karşı önerir: örnekler yalnız birkaç durum birleşiminden birine sahipse, her birleşim için bir prototip saklayıp klonlamak elverişlidir. Hocanın araba için verdiği gerekçe ise Builder ile Prototype arasındaki seçime ilişkindir ve "sınırlı durum uzayı → Prototype uygun değil" sonucu slayttaki maddeyle çelişir.

Soru 4: Soyut sınıf alt sınıfları nasıl ayırt ediyor? (derste sözlü, 00:57:04)

Soru (bir öğrenci): Soyut bir ana sınıftan somut sınıflar türetiyoruz. Abstract class bu alt sınıfların ayrımını nerede yapıyor? Geçen derste switch-case vardı; burada geçiş nasıl sağlanıyor?

Cevap: Ayrımı Java derleyicisi ve çalışma zamanı kendisi yapar; bu çok biçimliliktir. Hocanın `Insan`, `Calisan`, `Ogrenci` örneğinde (bkz. Bölüm 7) `Insan` türünden bir dizideki her nesne, yazdırılırken kendi alt sınıfındaki `toString()` metoduyla kendini tanıttı. `Insan` soyut yapıldığında da sonuç değişmedi; derleyici kalıtım ilişkisini bilir, bytecode'u ona göre üretir ve çalışma anında doğru metodu çağırır.

Dersin Toparlaması

Bu derste yaratımsal kalıpların kalanı tamamlandı. Factory Method, oluşturulacak nesnenin sınıfına alt sınıfların karar vermesini sağlar: üst sınıf ortak adımları somut metotlarla yürütürken türe özgü nesneyi soyut (çoğu zaman protected) bir factory metodu üzerinden alır. Document/Application framework'ü, Figure/Manipulator ve e-ticaret sitesindeki Store/Order örnekleri bu yapının farklı kullanım alanlarını gösterdi. Builder, karmaşık bir nesnenin oluşturulmasını gösteriminden ayırır; Director, Builder'a hangi parçayı hangi sırayla oluşturacağını söyler ve bu yönüyle factory kalıplarından daha fazla denetim sağlar. RTF okuyucu ve araç bayisi örneklerinde switch-case'in (ya da if-else zincirinin) problemin doğasında kaldığı, asıl kusurun alt sınıflarda anlamsız, boş gövdeli metotlar olduğu görüldü.

Prototype, yeni nesneleri bir prototipin klonlanmasıyla oluşturur. Oluşturulacak sınıflar çalışma anında belirleniyorsa ya da ürün hiyerarşisine paralel bir fabrika hiyerarşisinden kaçınılmak isteniyorsa tercih edilir. Klonlamada sığ ve derin kopya ayrımına, derin kopyada döngüsel ilişkilerin yol açabileceği sorunlara dikkat edilmelidir. Singleton ise bir sınıfın tek örneğinin yönetimini o sınıfın kendisine verir: statik bir örnek, private kurucu ve statik bir erişim metodu. Hoca bütün bu kalıplar için en önemli becerinin benzer kalıplar arasındaki farkları bilmek olduğunu vurguladı. Hocanın belirttiğine göre sıradaki konu büyük olasılıkla yapısal (structural) kalıplardır ve ilk kalıp Adapter olacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.